The RAG Framework and Its Components

Hello and welcome to the second lecture of this course on RAG tuning an LLM model. In this lecture, you will learn about the RAG framework and its components. You will also learn how to use RAG models from Hugging Face Transformers library, which is a popular and easy-to-use library for natural language processing.

As we learned in the previous lecture, RAG stands for **Retrieval-Augmented Generation**, which is a method that combines the strengths of both retrieval-based and generative approaches for natural language processing. Retrieval-based approaches use a large collection of documents or knowledge sources to find relevant information for a given query or context. Generative approaches use a large neural network or language model to generate natural language from scratch or based on some input. RAG combines these two approaches by using a retrieval system to select relevant documents from a knowledge source and then using a generative model to produce natural language that incorporates information from the retrieved documents.

The RAG framework consists of three main components: a **query encoder**, a **retriever**, and a **generator**. Let's look at each of these components in more detail.

- The query encoder is a neural network that encodes the input query or context into a vector representation. The query encoder can be any pre-trained language model, such as BERT, RoBERTa, or T5. The query encoder is responsible for capturing the meaning and intent of the input and producing a compact and informative representation that can be used for retrieval and generation.
- The retriever is a system that uses the query vector to search for and retrieve relevant documents from a knowledge source. The retriever can be based on different methods, such as exact match, TF-IDF, BM25, or dense vector similarity. The retriever is responsible for finding and ranking the most informative and relevant documents that can provide additional knowledge and facts for the input query or context.
- The generator is a neural network that decodes the query vector and the retrieved documents into natural language output. The generator can be any pre-trained sequence-to-sequence model, such as BART, T5, or GPT-3. The generator is responsible for generating fluent and coherent natural language that answers the query or continues the context, while incorporating information from the retrieved documents.

The RAG framework can be implemented in different ways, depending on how the retriever and the generator interact with each other. There are two main variants of RAG: RAG-Token and RAG-Sequence. Let's see what are the differences between them.

- RAG-Token is a variant of RAG that retrieves documents for each token generated by the generator. In other words, the retriever is called at every decoding step, and the retrieved documents are used to compute the token probabilities. This allows the generator to dynamically adapt to the generated tokens and retrieve relevant documents

accordingly. RAG-Token is suitable for tasks that require fine-grained and diverse information retrieval, such as question answering or fact verification.

- RAG-Sequence is a variant of RAG that retrieves documents for the whole input query or context. In other words, the retriever is called only once, before the decoding starts, and the retrieved documents are used to compute the sequence probabilities. This allows the generator to use a fixed set of documents for the whole output generation. RAG-Sequence is suitable for tasks that require coarse-grained and consistent information retrieval, such as text summarization or dialogue generation.

The RAG framework is a powerful and flexible method for retrieving and generating natural language with large language models. However, implementing RAG from scratch can be challenging and time-consuming, as it requires building and integrating different components and models. Fortunately, there is an easier way to use RAG, thanks to the Hugging Face Transformers library.

Hugging Face Transformers is a popular and easy-to-use library for natural language processing that provides access to hundreds of pre-trained models and pipelines. Among these models, there are several RAG models that are ready to use for different tasks and domains. You can use these RAG models with just a few lines of code, without having to worry about the details of the RAG framework. You can also fine-tune these RAG models on your own data and tasks, as we will see in the next section of this course.

To use a RAG model from Hugging Face Transformers, you need to install the library and import the necessary modules. You can do this by running the following commands in your terminal or notebook:

```
pip install transformers
from transformers import RagTokenizer, RagRetriever, RagTokenForGeneration,
RagSequenceForGeneration
```

Then, you need to choose which RAG model you want to use. There are several RAG models available, each with different query encoders, retrievers, and generators. You can find the list of RAG models on the Hugging Face model hub. For this lecture, we will use the `facebook/rag-token-nq` model, which is a RAG-Token model that uses DPR as the retriever and BART as the generator, and is fine-tuned on the Natural Questions dataset.

To use the `facebook/rag-token-nq` model, you need to instantiate the tokenizer, the retriever, and the model. You can do this by running the following code:

```
tokenizer = RagTokenizer.from_pretrained("facebook/rag-token-nq")
retriever = RagRetriever.from_pretrained("facebook/rag-token-nq", index_name="exact",
use_dummy_dataset=True)
model = RagTokenForGeneration.from_pretrained("facebook/rag-token-nq", retriever=retriever)
```

Note that we are using the `exact` index for the retriever, which means that the retriever will use exact match to search for documents in the knowledge source. The knowledge source for this model is the English Wikipedia, which is stored in a local database. We are also using the `use_dummy_dataset` argument to avoid downloading the whole Natural Questions dataset, which is not needed for inference.

Once you have instantiated the tokenizer, the retriever, and the model, you can use them to generate natural language output for any input query. For example, you can ask the model a question like “Who is the president of France?” and get an answer like “Emmanuel Macron”. You can do this by running the following code:

```
question = "Who is the president of France?"
input_ids = tokenizer(question, return_tensors="pt").input_ids
output = model.generate(input_ids)
answer = tokenizer.batch_decode(output, skip_special_tokens=True)[0]
print(answer)
```

The output of this code is:

```
Emmanuel Macron
```

As you can see, the model was able to answer the question correctly by retrieving and generating information from the knowledge source. You can also see the documents that the model retrieved for the question by running the following code:

```
docs = retriever(input_ids.numpy(), return_tensors="pt")
doc_titles = tokenizer.batch_decode(docs["doc_ids"], skip_special_tokens=True)
print(doc_titles)
```

The output of this code is:

```
['Emmanuel Macron', 'President of France', 'List of presidents of France', 'France', 'French presidential election, 2017']
```

As you can see, the model retrieved five documents that are relevant to the question, and used them to compute the token probabilities. You can also see the token probabilities by running the following code:

```
probs = model(input_ids, labels=output, return_dict=True).logits
probs = probs.softmax(dim=-1)
top_tokens = tokenizer.batch_decode(probs[0, -1].topk(5).indices)
print(top_tokens)
```

The output of this code is:

['Emmanuel Macron', 'François Hollande', 'Nicolas Sarkozy', 'Jacques Chirac', 'Charles de Gaulle']

As you can see, the model assigned the highest probability to the correct answer, “Emmanuel Macron”, and also considered some other possible answers that are former presidents of France.

This is how you can use a RAG model from Hugging Face Transformers to retrieve and generate natural language with large language models. You can experiment with different RAG models, input queries, and output formats to see how they work and what they produce. You can also fine-tune these RAG models on your own data and tasks, as we will see in the next section of this course.